

LLM Fundamentals

Module 8 — AI Engineering · Notebook 27

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

2026-07-22

LLM Fundamentals

LLM Fundamentals
Module 8 — AI Engineering · Notebook 27

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

Contents

- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

2026-07-22

LLM Fundamentals

└ Contents

- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

Outline

- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

2026-07-22

LLM Fundamentals
└ Tokens
└ Outline

What is a Large Language Model?

A next-token predictor

An LLM learns statistical relationships between words from massive corpora. Given some text, it computes the *most probable continuation*, one token at a time.

Analogy

Someone who has read every book ever written: they can continue any topic plausibly — but have *no built-in sense* of what is true, current, or private to your company.

2026-07-22

LLM Fundamentals

└ Tokens

└ What is a Large Language Model?

~6 min — the framing for the whole lecture.

Point: everything surprising about LLMs (hallucination, cutoff, prompt sensitivity) follows from *one* fact — it predicts the next token. Say that now; you will call back to it on every remaining slide.

Ask the room: “If it only predicts likely text, why does it ever get facts right?” Answer: true statements are common in the training data, so plausibility and truth correlate — but only correlate. That gap is the entire second half of this lecture.

Resist defining tokens, attention or temperature here — they each get their own slide.

A next-token predictor

An LLM learns statistical relationships between words from massive corpora. Given some text, it computes the *most probable continuation*, one token at a time.

Analogy

Someone who has read every book ever written: they can continue any topic plausibly — but have no *built-in* sense of what is true, current, or private to your company.

How LLMs see text: tokens

ChatGPT  = 3 tokens

The cat sat.  = 4 tokens

- Cost and the **context window** are measured in tokens — not characters, not words.
- Rule of thumb: 1 English token $\approx$ 4 characters $\approx$ 0.75 words (1,000 tokens $\approx$ 750 words).
- Code and non-English text tokenize *less* efficiently; the model is blind to characters *within* a token (“how many r’s in strawberry?”).

2026-07-22

LLM Fundamentals

Tokens

How LLMs see text: tokens

How LLMs see text: tokens

ChatGPT  = 3 tokens

The cat sat.  = 4 tokens

- Cost and the **context window** are measured in tokens — not characters, not words.
- Rule of thumb: 1 English token $\approx$ 4 characters $\approx$ 0.75 words (1,000 tokens $\approx$ 750 words).
- Code and non-English text tokenize *less* efficiently; the model is blind to characters *within* a token (“how many r’s in strawberry?”).

~10 min. Point: tokens are the unit of *cost*, *context* and *blindness*.

The strawberry example is the one they’ll remember — the model can’t count letters it never sees separately. Ask it live if you have a provider configured; it fails convincingly.

Ask the room: “Your prompt is 2,000 words. Roughly how many tokens, and does it fit?” ($\approx$ 2,700 — fits anywhere today.) The arithmetic matters because they’ll be paying per token in Module 8.

Non-English matters here: German compounds tokenize badly, so the same text costs noticeably more than in English. Worth naming for a German-speaking cohort.

What are parameters? Why “large”?

Model	Parameters	Year
GPT-2	1.5 billion	2019
Llama-3 (open-weights)	8B / 70B / 405B	2024
Frontier (GPT-5.x, Claude, Gemini)	hundreds of billions (est.)	2023+

Key insight

There is *no* manual list of facts inside the model. Knowledge is *implicitly distributed* across all the weights — which is exactly why it can be confidently wrong.

2026-07-22

LLM Fundamentals

- └ Tokens

- └ What are parameters? Why “large”?

~7 min. Point: parameters are learned weights, not stored facts. There is no row in a table saying “Paris is the capital of France.”

That is *why* you cannot patch a wrong fact by editing something, and why the fix is RAG (retrieval), not surgery on the model.

Keep the numbers loose — frontier parameter counts are estimates and they date fast. The trend is the teaching point, not the figures.

Ask the room: “Where inside the model is the fact stored?” The honest answer — smeared across billions of weights, nowhere in particular — is genuinely unsettling and should be.

What are parameters? Why “large”?

Model	Parameters	Year
GPT-2	1.5 billion	2019
Llama-3 (open-weights)	8B / 70B / 405B	2024
Frontier (GPT-5.x, Claude, Gemini)	hundreds of billions (est.)	2023+

Key insight

There is no manual list of facts inside the model. Knowledge is implicitly distributed across all the weights — which is exactly why it can be confidently wrong.

Outline

- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

2026-07-22

LLM Fundamentals

- └ Training
 - └ Outline

Outline

- Tokens
- Training
- Transformer
- Using LLMs
- Limits

The training objective: next-token prediction

Hide → predict → measure → nudge, trillions of times

Hide the next token, ask the model to predict it, measure the error, nudge every parameter slightly toward a better prediction.

$$\mathcal{L} = - \sum_i y_i \log \hat{p}_i$$

where $y_i = 1$ for the one true next token and 0 otherwise (a *one-hot* target), and $\hat{p}_i$ is the model's predicted probability for token i . Grammar, facts, and reasoning emerge as *side-effects*.

You write this exact loop — loss → backward → step — yourself in **Module 6 (PyTorch)**; an LLM trains with the same five moves, just with more parameters and more data.

2026-07-22

LLM Fundamentals

└ Training

└ The training objective: next-token prediction

~**10 min.** Point: the objective is embarrassingly simple, and everything impressive is a *side-effect* of doing it at scale.

Don't derive the cross-entropy. Read it in words: "the loss is small when the probability assigned to the token that actually came next is large." That sentence is the whole equation.

The call-forward matters: if your cohort is doing Module 6, promise them they will write this exact loop in PyTorch. It demystifies the whole thing — same five moves, more data.

Misconception to kill: that the model was "taught" grammar or facts. Nobody labelled anything — the text was its own label.

The training objective: next-token prediction

Hide → predict → measure → nudge, trillions of times

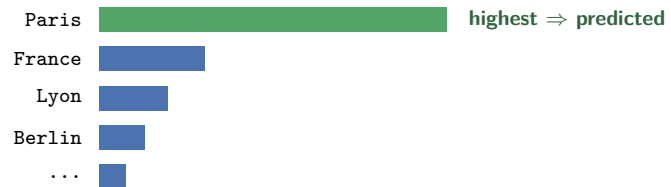
Hide the next token, ask the model to predict it, measure the error, nudge every parameter slightly toward a better prediction.

$$\mathcal{L} = - \sum_i y_i \log \hat{p}_i$$

where $y_i = 1$ for the one true next token and 0 otherwise (a one-hot target), and $\hat{p}_i$ is the model's predicted probability for token i . Grammar, facts, and reasoning emerge as side-effects. You write this exact loop — loss → backward → step — yourself in **Module 6 (PyTorch)**; an LLM trains with the same five moves, just with more parameters and more data.

The output is a probability over *every* token

context: "The capital of France is"



The model never returns “the answer” — it returns a *distribution*. temperature controls how often it picks something other than the top token.

2026-07-22

LLM Fundamentals

Training

The output is a probability over *every* token

~8 min. Point: the output is a *distribution*, not an answer. This is the slide that explains non-determinism.

Ask the room: “You send the same prompt twice and get different answers. Bug or expected?” Expected — and controllable. temperature 0 takes the top token every time; higher temperature samples further down this bar chart.

Tie it to practice: **temperature 0 for extraction and classification** (you want the same answer twice), higher only when you want variety. They will set this parameter constantly from NB 28 onward.

Note that even at temperature 0 providers aren’t perfectly reproducible — batching and hardware introduce jitter. Mention only if asked; it’s a rabbit hole.

The output is a probability over every token



The model never returns “the answer” — it returns a *distribution*. temperature controls how often it picks something other than the top token.

Outline

- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

2026-07-22

LLM Fundamentals
└ Transformer
 └ Outline

- Tokens
- Training
- **Transformer**
- Using LLMs
- Limits

The architecture: the Transformer

A stack of attention + feed-forward layers

Each token becomes a vector; each layer lets tokens *exchange information* (attention), then transforms each vector (feed-forward). Stack many layers → rich, context-aware representations.

- All tokens are processed *in parallel* during training — a huge speed-up over older recurrent models.
- Depth (layers) + width (vector size) + data = the “large” in LLM.

LLM Fundamentals

Transformer

The architecture: the Transformer

~7 min. Point: a stack of layers where tokens repeatedly *look at each other* and then get transformed.

Pitch this at intuition level — the next slide has the mechanics. The one structural fact worth stressing is **parallelism**: unlike the recurrent models it replaced, all tokens are processed at once, which is precisely what made training at this scale affordable.

If the room is non-technical, stop here and skip the Q/K/V slide entirely — it is safely optional, and nothing later in the course depends on it.

A stack of attention + feed-forward layers

Each token becomes a vector; each layer lets tokens *exchange information* (attention), then transforms each vector (feed-forward). Stack many layers → rich, context-aware representations.

- All tokens are processed *in parallel* during training — a huge speed-up over older recurrent models.
- Depth (layers) + width (vector size) + data = the “large” in LLM.

The Attention mechanism (Q / K / V)

Step	What happens
1. Project	compute Query, Key, Value for every token via learned matrices
2. Score	$score_{ij} = Q_i \cdot K_j / \sqrt{d_k}$
3. Normalise	softmax over $j \rightarrow$ weights summing to 1
4. Mix	new vector = $\sum_j weight_{ij} V_j$

d_k is the length of each Key/Query vector; dividing by $\sqrt{d_k}$ keeps scores from blowing up. **Intuition:** for each word, look at every other word and decide how much to listen to it.

2026-07-22

LLM Fundamentals

- Transformer

- The Attention mechanism (Q / K / V)

~12 min — the most technical slide in the deck. Optional; cut it first if you are short on time.

Lead with the intuition line at the bottom, *then* the table — not the other way round.
The analogy that works: Query = “what am I looking for?”, Key = “what do I offer?”, Value = “what I actually contribute.” Scoring Q against K is a relevance match; the softmax turns relevance into a weighting; the mix is a weighted average.

Worked example on the board: in “The animal didn’t cross the street because *it* was too tired”, ask what “it” should attend to. Attention is what resolves that — and the answer flips to “street” if you change “tired” to “wide”.

Don’t defend the $\sqrt{d_k}$; it’s numerical hygiene, not insight.

The Attention mechanism (Q / K / V)

Step	What happens
1. Project	compute Query, Key, Value for every token via learned matrices
2. Score	$score_{ij} = Q_i \cdot K_j / \sqrt{d_k}$
3. Normalise	softmax over $j \rightarrow$ weights summing to 1
4. Mix	new vector = $\sum_j weight_{ij} V_j$

d_k is the length of each Key/Query vector; dividing by $\sqrt{d_k}$ keeps scores from blowing up. **Intuition:** for each word, look at every other word and decide how much to listen to it.

Outline

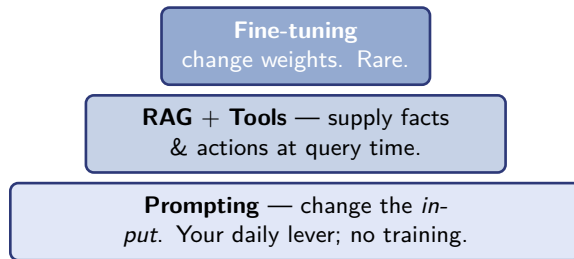
- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

2026-07-22

LLM Fundamentals
└ Using LLMs
 └ Outline

- Tokens
- Training
- Transformer
- Using LLMs
- Limits

Three levels of using an LLM



Fine-tuning vs RAG

Fine-tuning changes *behaviour* / *style*. **RAG** (NB 29) keeps the weights frozen and supplies *fresh facts* — don't fine-tune to add knowledge.

2026-07-22

LLM Fundamentals

└ Using LLMs

└ Three levels of using an LLM

Three levels of using an LLM



Fine-tuning vs RAG

Fine-tuning changes *behaviour* / *style*. **RAG** (NB 29) keeps the weights frozen and supplies *fresh facts* — don't fine-tune to add knowledge.

~8 min. Point: the pyramid is deliberately upside-down in effort — the widest, cheapest lever (prompting) is the one you'll use 95 % of the time.

The single most valuable correction in this lecture: “we'll fine-tune it on our documents” is almost always wrong. Fine-tuning teaches *behaviour and format*; it is a poor and expensive way to install *facts*, which go stale the day after you train. Facts belong in retrieval.

Ask the room: “You want the bot to answer from your 400-page handbook. Which level?” RAG — and if anyone says fine-tune, this is the moment to unpick it.

Forward-reference NB 29 (RAG) and NB 30 (tools) so it lands as a roadmap, not a taxonomy.

Five prompting techniques

Technique	When to reach for it
Zero-shot	a general question; you trust the defaults
Few-shot	you want a <i>consistent</i> format — show 2–3 examples
Chain-of-Thought	multi-step reasoning — ask it to work step by step
Role assignment	set tone / depth — “you are a senior analyst. . .”
Format constraint	machine-parseable output — “return JSON with keys. . .”

A production prompt stacks several: *role* + *context* + *task* + *format* + *constraints*.

2026-07-22

LLM Fundamentals

- Using LLMs

- Five prompting techniques

~10 min — make this the interactive one.

Don’t lecture the table. Take a deliberately bad prompt (“summarise this”) and improve it live with the room, adding one technique at a time until it’s role + context + task + format + constraints. They should watch the output get sharper at each step.

Format constraint is the one with engineering consequences: “return JSON with keys x, y” is what makes an LLM callable as a *function* rather than a chat toy — that’s the whole premise of NB 28.

Honest caveat on chain-of-thought: on newer reasoning models, explicitly asking for step-by-step adds less than it used to — they already do it internally.

Technique	When to reach for it
Zero-shot	a general question; you trust the defaults
Few-shot	you want a <i>consistent</i> format — show 2–3 examples
Chain-of-Thought	multi-step reasoning — ask it to work step by step
Role assignment	set tone / depth — “you are a senior analyst. . .”
Format constraint	machine-parseable output — “return JSON with keys. . .”

A production prompt stacks several: *role* + *context* + *task* + *format* + *constraints*.

Outline

- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

2026-07-22

LLM Fundamentals
└ Limits
└ Outline

- Tokens
- Training
- Transformer
- Using LLMs
- Limits

Two structural limitations

1. Hallucinations

The model predicts *plausible* continuations, not *true* ones — so it can invent clauses, statistics, or APIs with full confidence.

2. Knowledge cutoff

It knows nothing after its training date, and nothing about your *private* data — no current prices, no internal reports.

Mitigations (built later in the course)

RAG (NB 29) grounds answers in evidence · **Tools** (NB 30) let it look things up · **Evaluation** (NB 32) measures how often it's wrong.

2026-07-22

LLM Fundamentals

└ Limits

└ Two structural limitations

~**10 min + close**. Point: both limitations are *structural*, not bugs awaiting a patch — they fall straight out of “predicts plausible continuations.” Close the loop with slide 1 explicitly.

Frame hallucination precisely: the model is not lying and does not know it is wrong. Nothing in the objective ever rewarded truth — only plausibility.

The professional habit to leave them with: never ask “is it accurate?” — ask “*how would I measure* how often it's wrong, and what happens when it is?” That reframing is the bridge to NB 32 and to Module 18.

Send them to Notebook 27 — it runs entirely offline against the MockLLM, so nobody needs an API key to start.

1. Hallucinations

The model predicts *plausible* continuations, not *true* ones — so it can invent clauses, statistics, or APIs with full confidence.

2. Knowledge cutoff

It knows nothing after its training date, and nothing about your *private* data — no current prices, no internal reports.

Mitigations (built later in the course)

RAG (NB 29) grounds answers in evidence · **Tools** (NB 30) let it look things up · **Evaluation** (NB 32) measures how often it's wrong.